Test Case 1

3. Hospital Billing and Patient Classification Using Python

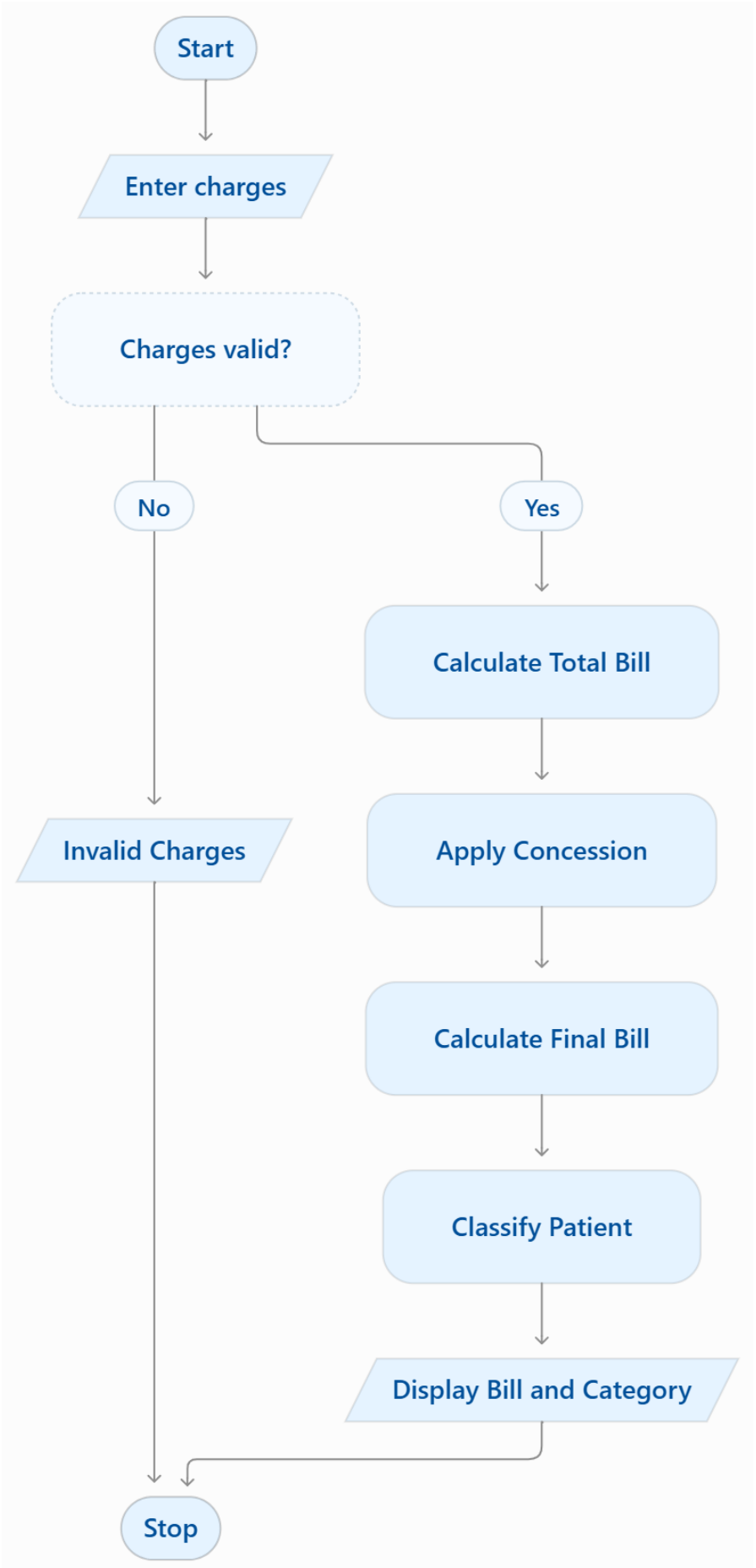
1. Problem Statement

3. Hospital Billing and Patient Classification: Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a supplied payment policy.

2. Algorithm

1. Start the program.
2. Read consultation, diagnostic, and treatment charges.
3. Check whether any charge is negative.
4. If a charge is negative, display an invalid billing message and stop.
5. Calculate the total bill by adding all three charges.
6. If the total bill is ₹10,000 or more, calculate a 10% concession.
7. Limit the concession to a maximum of ₹2,000.
8. Calculate the final bill by subtracting the concession from the total bill.
9. Classify the patient according to the final payable amount.
10. Display the total bill, concession, final amount, and patient classification.
11. Stop the program.

3. Flowchart


Start
Enter charges
Charges valid?
No
Yes
Invalid Charges
Calculate Total Bill
Apply Concession
Calculate Final Bill
Classify Patient
Display Bill and Category
Stop

4.Source code

```
consultation = float(input("Enter consultation charges: ₹"))
diagnostic = float(input("Enter diagnostic charges: ₹"))
treatment = float(input("Enter treatment charges: ₹"))
if consultation < 0 or diagnostic < 0 or treatment < 0:
    print("Invalid charges! Charges cannot be negative.")
else:
    total_bill = consultation + diagnostic + treatment
    if total_bill >= 10000:
        concession = total_bill * 0.10
        concession = min(concession, 2000)
    else:
        concession = 0
    final_bill = total_bill - concession
    if final_bill < 5000:
        category = "Low Payment"
    elif final_bill <= 15000:
        category = "Medium Payment"
    else:
        category = "High Payment"
    print("\n----- HOSPITAL BILL -----")
    print(f"Consultation Charges: ₹{consultation:.2f}")
    print(f"Diagnostic Charges:   ₹{diagnostic:.2f}")
    print(f"Treatment Charges:    ₹{treatment:.2f}")
    print(f"Total Bill:           ₹{total_bill:.2f}")
    print(f"Concession:           ₹{concession:.2f}")
    print(f"Final Payable Amount: ₹{final_bill:.2f}")
    print(f"Patient Classification: {category}")
```

5. Sample Input

```
Enter consultation charges: ₹1500
Enter diagnostic charges: ₹2500
Enter treatment charges: ₹8000
```

6. Sample Output

```
----- HOSPITAL BILL -----
Consultation Charges: ₹1500.00
Diagnostic Charges:   ₹2500.00
Treatment Charges:    ₹8000.00
Total Bill:           ₹12000.00
Concession:           ₹1200.00
Final Payable Amount: ₹10800.00
Patient Classification: Medium Payment
```

Test case 2

8. Manufacturing Cost and Profitability Analysis Using Python

1. Problem Statement

8.Write a Python program to calculate the total manufacturing cost by considering material costs, labour charges, wastage costs, and machine costs. Calculate the profit using the selling price, determine the profit percentage, and check whether the production meets the specified profit target.

2. Algorithm

1. Start.
2. Enter material cost, labour charges, wastage cost, machine cost, selling price, and profit target percentage.
3. Validate that all input values are non-negative.

4. Calculate total manufacturing cost.
5. Calculate profit by subtracting manufacturing cost from selling price.
6. Calculate profit percentage.
7. Compare the profit percentage with the target percentage.
8. Display the total cost, profit, profit percentage, and result.
9. Stop.

3. Flowchart



4.Source Code

```
material = float(input("Enter material cost: "))
labour = float(input("Enter labour charges: "))
wastage = float(input("Enter wastage cost: "))
machine = float(input("Enter machine cost: "))
selling_price = float(input("Enter selling price: "))
target = float(input("Enter target profit percentage: "))
if min(material, labour, wastage, machine,
       selling_price, target) < 0:
    print("Invalid input! Values cannot be negative.")
else:
    total_cost = material + labour + wastage + machine
    profit = selling_price - total_cost
    if total_cost > 0:
        profit_percent = (profit / total_cost) * 100
    else:
        profit_percent = 0
    print("\n--- Manufacturing Report ---")
    print("Total Manufacturing Cost:", total_cost)
    print("Selling Price:", selling_price)
    print("Profit/Loss:", profit)
    print("Profit Percentage:", round(profit_percent, 2), "%")
    if profit_percent >= target:
        print("Result: Profit Target Achieved")
    else:
        print("Result: Profit Target Not Achieved")
```

5.Sample Input

```
Enter material cost: 5000
Enter labour charges: 2000
Enter wastage cost: 500
Enter machine cost: 1500
Enter selling price: 12000
Enter target profit percentage: 30
```

6. Sample Output

```
--- Manufacturing Report ---
Total Manufacturing Cost: 9000.0
Selling Price: 12000.0
Profit/Loss: 3000.0
Profit Percentage: 33.33 %
Result: Profit Target Achieved
```

Test Case 3

15. Smart Parking Fee Management Using Python

1. Problem Statement

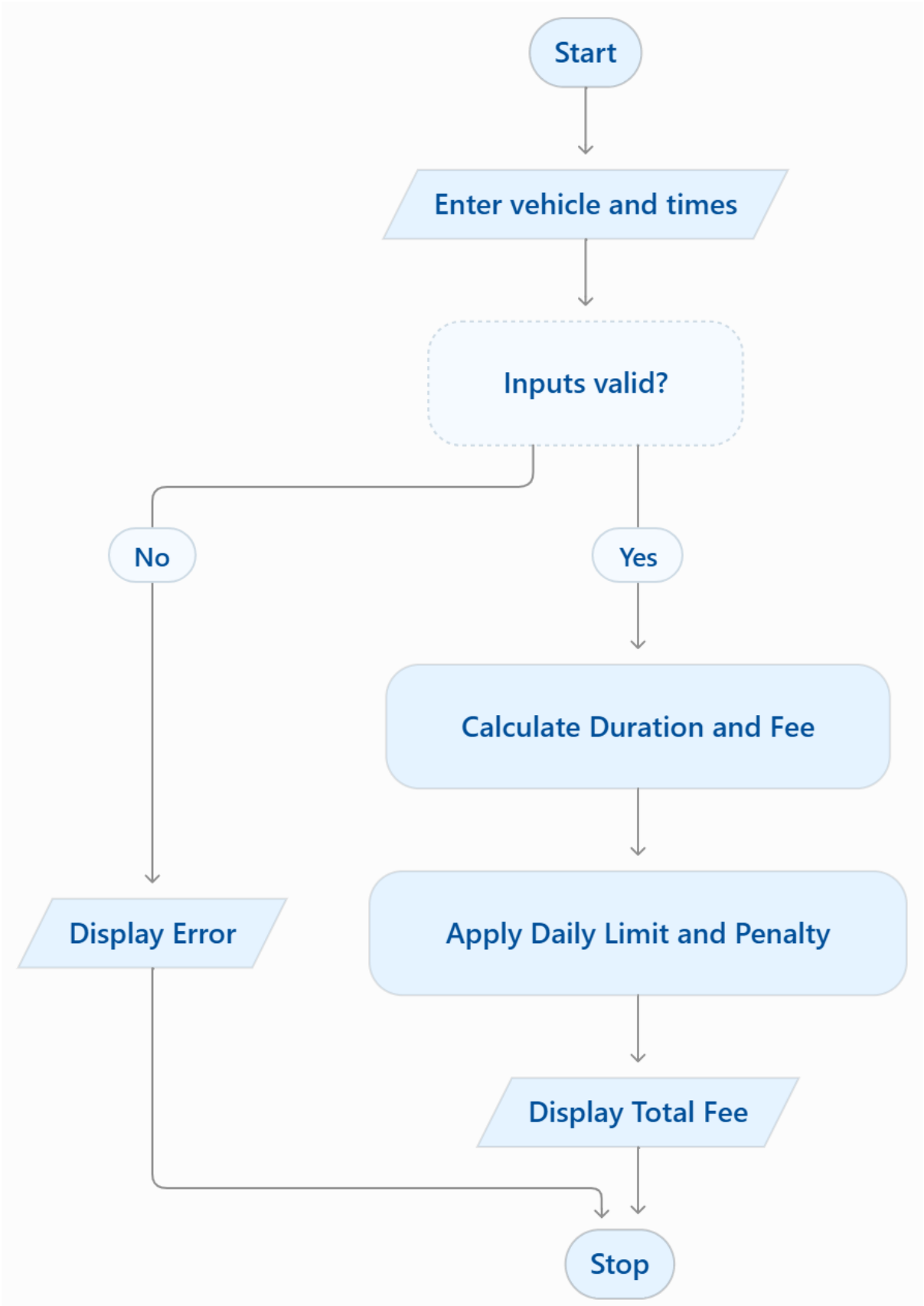
15.Write a Python program to calculate parking fees based on vehicle category, entry time, exit time, overnight parking, daily fee limits, and penalty conditions. Validate the time inputs and display the total parking fee.

2. Algorithm

1. Start.
2. Enter the vehicle category, entry time, and exit time.
3. Validate the vehicle category and time format.
4. Calculate the parking duration, including overnight parking.

5. Calculate the charge based on the vehicle category and started hours.
6. Apply the daily maximum limit.
7. Add the penalty if parking exceeds 24 hours.
8. Display parking duration and total fee.
9. Stop.

3. Flowchart



4.Source code

```
from datetime import datetime
from math import ceil
rates = {
    "two-wheeler": 20,
    "car": 40,
    "heavy": 60
}
daily_limits = {
    "two-wheeler": 200,
    "car": 400,
    "heavy": 600
}
try:
    vehicle = input("Enter vehicle type (two-wheeler/car/heavy): ").lower()
    entry = input("Enter entry date and time (YYYY-MM-DD HH:MM): ")
    exit_time = input("Enter exit date and time (YYYY-MM-DD HH:MM): ")
    if vehicle not in rates:
        print("Invalid vehicle category!")
    else:
        entry_time = datetime.strptime(entry, "%Y-%m-%d %H:%M")
        exit_datetime = datetime.strptime(exit_time, "%Y-%m-%d %H:%M")
        if exit_datetime <= entry_time:
            print("Invalid time! Exit must be after entry.")
        else:
            minutes = (exit_datetime - entry_time).total_seconds() / 60
            hours = ceil(minutes / 60)
            full_days = hours // 24
            remaining_hours = hours % 24
```

```
        rate = rates[vehicle]
        daily_limit = daily_limits[vehicle]
        fee = (full_days * daily_limit) + min(
            remaining_hours * rate, daily_limit
        )
        penalty = 100 if hours > 24 else 0
        total = fee + penalty
        print("\n--- PARKING BILL ---")
        print("Vehicle Type:", vehicle)
        print("Parking Duration:", hours, "hours")
        print("Parking Fee: Rs.", fee)
        print("Penalty: Rs.", penalty)
        print("Total Parking Fee: Rs.", total)
except ValueError:
    print("Invalid date or time format!")
```

5.Sample Input

```
Enter vehicle type (two-wheeler/car/heavy): car
Enter entry date and time (YYYY-MM-DD HH:MM): 2026-10-28 22:00
Enter exit date and time (YYYY-MM-DD HH:MM): 2026-10-29 02:30
```

6.Sample Output

```
--- PARKING BILL ---
Vehicle Type: car
Parking Duration: 5 hours
Parking Fee: Rs. 200
Penalty: Rs. 0
Total Parking Fee: Rs. 200
```

Test Case 4

33. Campus Registration Identifier Analyzer Using Python

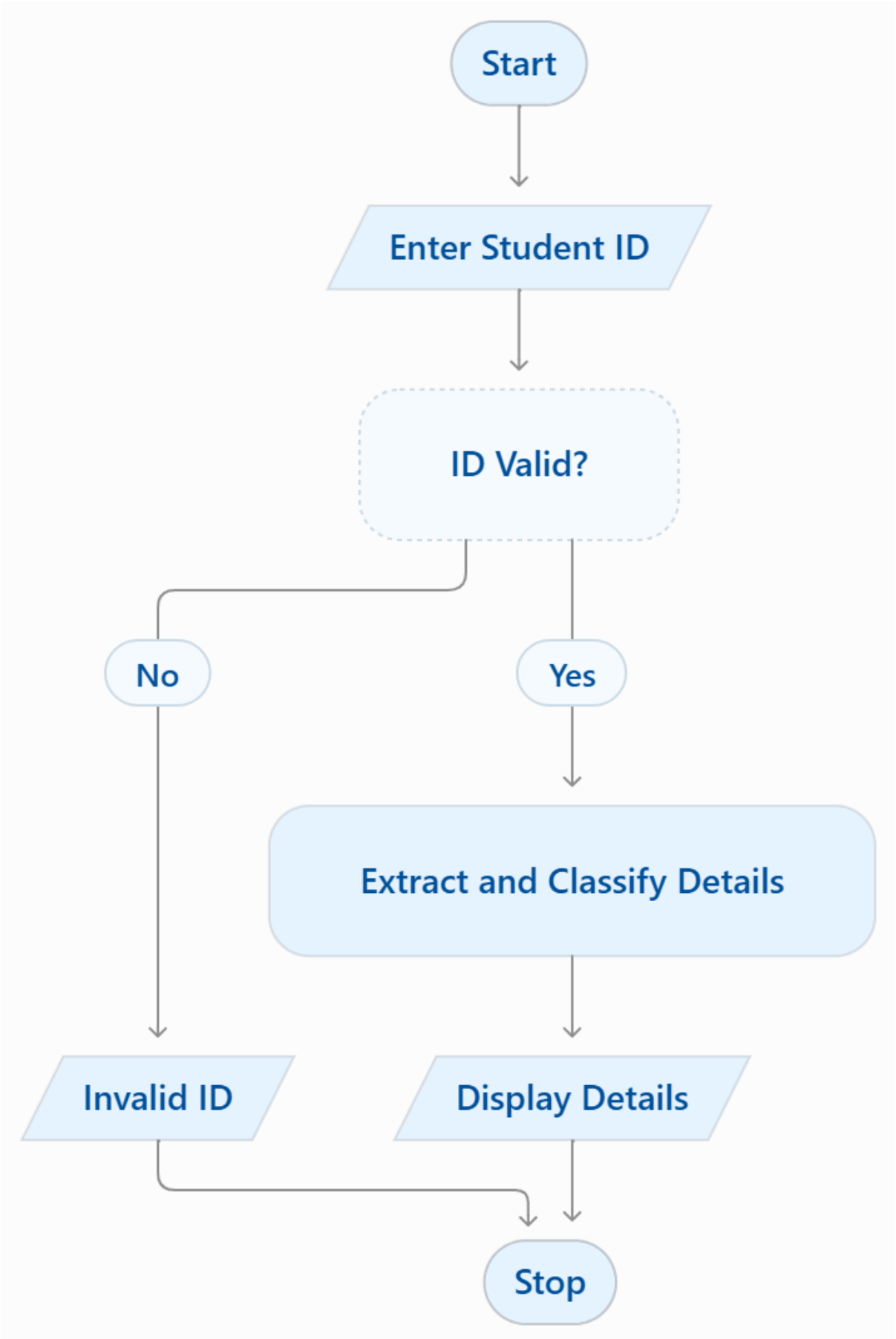
1. Problem Statement

33.Write a Python program to analyze student registration IDs, validate their format, extract the admission year, branch code, and student number, and classify students according to their branch and admission year.

2. Algorithm

1. Start.
2. Enter the student registration ID.
3. Check whether the ID contains exactly nine characters.
4. Validate the admission year, branch code, and student number.
5. If invalid, display an error message.
6. If valid, extract the year, branch, and student number.
7. Classify the student by branch and admission year.
8. Display the extracted details and classification.
9. Stop.

3. Flowchart


Start
Enter Student ID
ID Valid?
No
Yes
Extract and Classify Details
Invalid ID
Display Details
Stop

4.Source code

```
student_id = input("Enter Student ID (YYBBBSSSS): ").upper()
branches = ["CSE", "ECE", "EEE"]
if (len(student_id) == 9
   and student_id[:2].isdigit()
   and student_id[2:5] in branches
   and student_id[5:].isdigit()):
   year = "20" + student_id[:2]
   branch = student_id[2:5]
   number = student_id[5:]
   print("\n--- STUDENT DETAILS ---")
   print("Admission Year:", year)
   print("Branch:", branch)
   print("Student Number:", number)
   print("Classification:", branch, year, "Batch")
else:
   print("Invalid Student ID!")
```

5.Sample Input

```
Enter Student ID (YYBBBSSSS): 26CSE0196
```

6. Sample Output

```
--- STUDENT DETAILS ---
Admission Year: 2026
Branch: CSE
Student Number: 0196
Classification: CSE 2026 Batch
```

Test Case 5

40. Traffic Flow and Congestion Analysis Using Python

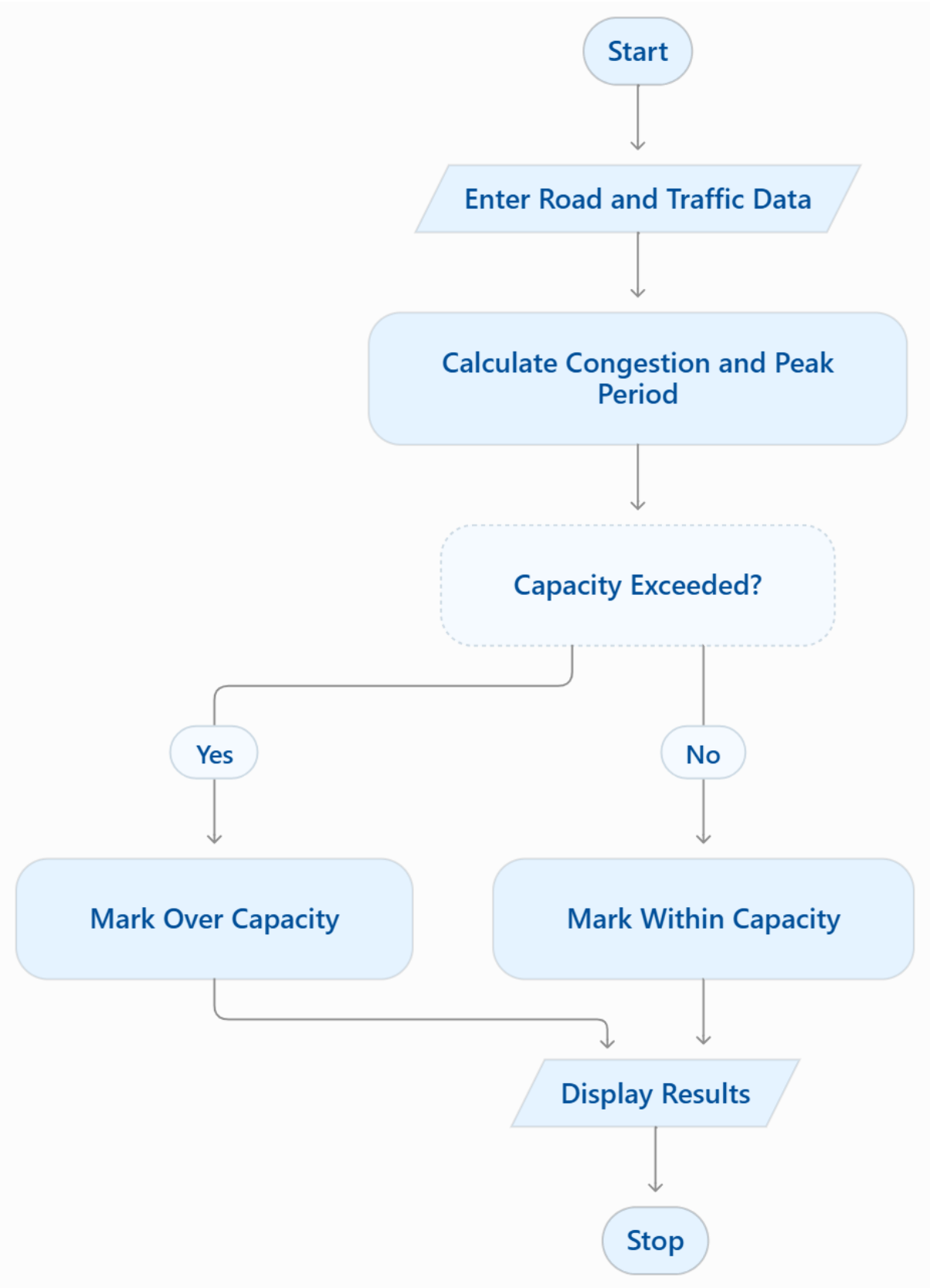
1. Problem Statement

40.Write a Python program to analyze vehicle counts across multiple roads and time intervals. Identify the peak traffic period, classify congestion levels, and determine which roads exceed their traffic capacity.

2. Algorithm

1. Start.
2. Enter the number of roads and time intervals.
3. Enter each road's name, capacity, and vehicle counts for each interval.
4. Calculate congestion percentage for each interval.
5. Classify congestion as low, moderate, or high.
6. Identify the peak traffic period.
7. Check whether each road exceeds its capacity.
8. Display the results.
9. Stop.

3. Flowchart



4.Source Code

```
roads = int(input("Enter number of roads: "))
```

```
intervals = int(input("Enter number of time intervals: "))
if roads <= 0 or intervals <= 0:
    print("Invalid number of roads or intervals.")
else:
    times = input("Enter time intervals separated by spaces: ").split()
    if len(times) != intervals:
        print("Error: Number of time intervals does not match.")
    else:
        traffic_data = {}
        capacities = {}
        for i in range(roads):
            name = input("\nEnter road name: ")
            capacity = int(input("Enter road capacity: "))
            if capacity <= 0:
                print("Invalid capacity!")
                break
            counts = list(map(int, input(
                "Enter vehicle counts for each interval: "
            ).split()))

            if len(counts) != intervals or min(counts) < 0:
                print("Invalid vehicle counts!")
                break
            capacities[name] = capacity
            traffic_data[name] = counts
        else:
            print("\n--- TRAFFIC ANALYSIS ---")
            totals = [
```

```
        sum(traffic_data[road][j] for road in traffic_data)
        for j in range(intervals)
    ]
    peak = totals.index(max(totals))
    print("Peak Traffic Period:", times[peak])
    for road, counts in traffic_data.items():
        capacity = capacities[road]
        maximum = max(counts)
        percentage = maximum / capacity * 100
        if percentage < 50:
            level = "Low"
        elif percentage <= 80:
            level = "Moderate"
        else:
            level = "High"
        exceeded = any(c > capacity for c in counts)
        print("\nRoad:", road)
        print("Maximum Vehicles:", maximum)
        print("Congestion:", level)
        if exceeded:
            print("Status: Capacity Exceeded")
        else:
            print("Status: Within Capacity")
```

5. Sample Input

```
Enter number of roads: 2
Enter number of time intervals: 3
Enter time intervals separated by spaces: 9AM 1PM 6PM

Enter road name: Road A
Enter road capacity: 100
Enter vehicle counts for each interval: 30 60 90

Enter road name: Road B
Enter road capacity: 80
Enter vehicle counts for each interval: 20 50 100
```

6. Sample Output

```
--- TRAFFIC ANALYSIS ---
Peak Traffic Period: 6PM

Road: Road A
Maximum Vehicles: 90
Congestion: High
Status: Within Capacity

Road: Road B
Maximum Vehicles: 100
Congestion: High
Status: Capacity Exceeded
```

Test Case 6

52. Hackathon Team Formation and Ranking

1. Problem Statement

52.Write a Python program to form hackathon teams, validate team sizes, calculate total scores from multiple rounds, and rank teams based on their scores. If two teams have the same total score, use the highest score in the final round to decide the ranking.

2. Algorithm

1. Start the program.
2. Enter the number of teams and the minimum and maximum team sizes.
3. Enter each team's name, member count, and scores for three rounds.
4. Validate the team size.
5. Calculate each team's total score.
6. Sort teams by total score in descending order.
7. If totals are tied, compare final-round scores.
8. Display valid teams and their rankings.
9. Stop the program.

3. Flowchart


Start
Enter Team Details
Validate Team Size
Calculate Scores
Rank Teams and Resolve Ties
Display Rankings
Stop

4.Source Code

```
teams = []
n = int(input("Enter number of teams: "))
min_size = 2
max_size = 4
for i in range(n):
    print("\nTeam", i + 1)
    name = input("Enter team name: ")
    members = int(input("Enter number of members: "))
    if members < min_size or members > max_size:
        print("Invalid team size! Team rejected.")
        continue
    r1 = int(input("Enter Round 1 score: "))
    r2 = int(input("Enter Round 2 score: "))
    r3 = int(input("Enter Round 3 score: "))
    total = r1 + r2 + r3
    teams.append({
        "name": name,
        "members": members,
        "total": total,
        "final": r3
    })
teams.sort(key=lambda t: (t["total"], t["final"]), reverse=True)
print("\n--- Hackathon Rankings ---")
for rank, team in enumerate(teams, start=1):
    print(rank, team["name"],
          "| Members:", team["members"],
          "| Total Score:", team["total"],
```

"| Final Round:", team["final"])

5.Sample Input

```
Enter number of teams: 3

Team 1
Enter team name: Alpha
Enter number of members: 3
Enter Round 1 score: 80
Enter Round 2 score: 85
Enter Round 3 score: 90

Team 2
Enter team name: Beta
Enter number of members: 2
Enter Round 1 score: 85
Enter Round 2 score: 85
Enter Round 3 score: 95

Team 3
Enter team name: Gamma
Enter number of members: 5
```

6. Sample Output

```
Invalid team size! Team rejected.

--- Hackathon Rankings ---
1 Beta | Members: 2 | Total Score: 265 | Final Round: 95
2 Alpha | Members: 3 | Total Score: 255 | Final Round: 90
```

Test Case 7

74. Campus Festival Registration and Financial Audit Using Python

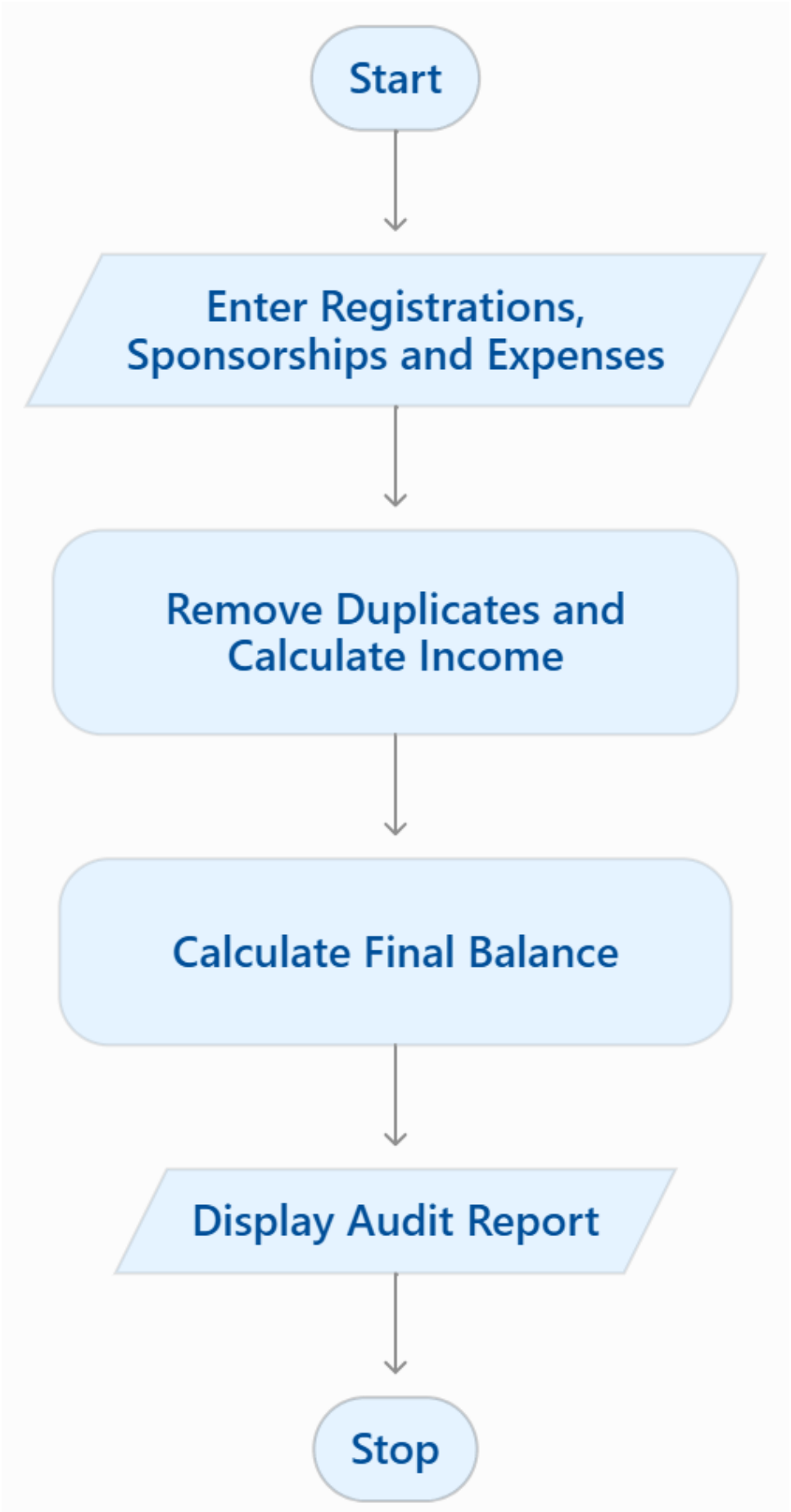
1. Problem Statement

74.Write a Python program to manage campus festival registrations and perform a financial audit. The program should maintain participant registration IDs, detect duplicate registrations, calculate total registration fees and sponsorship contributions, calculate total expenses, and determine the final financial balance.

2. Algorithm

1. Start.
2. Enter participant registration IDs.
3. Check for duplicate IDs and count valid participants.
4. Calculate total registration fees.
5. Enter sponsorship contributions and festival expenses.
6. Calculate total income and final financial balance.
7. Display valid participants, duplicate registrations, income, expenses, and balance.
8. Stop.

3. Flowchart


Start
Enter Registrations, Sponsorships and Expenses
Remove Duplicates and Calculate Income
Calculate Final Balance
Display Audit Report
Stop

4.Source Code

```
n = int(input("Enter number of registrations: "))
participants = []
duplicates = []
for i in range(n):
    reg_id = input("Enter registration ID: ")
    if reg_id in participants:
        duplicates.append(reg_id)
    else:
        participants.append(reg_id)
fee = 500
registration_income = len(participants) * fee
s = int(input("Enter number of sponsorships: "))
sponsorship = 0
for i in range(s):
    sponsorship += float(input("Enter sponsorship amount: "))
e = int(input("Enter number of expenses: "))
expenses = 0
for i in range(e):
    expenses += float(input("Enter expense amount: "))
total_income = registration_income + sponsorship
balance = total_income - expenses
print("\n--- FESTIVAL AUDIT REPORT ---")
print("Valid Participants:", len(participants))
print("Duplicate Registrations:", duplicates)
print("Registration Income: Rs.", registration_income)
print("Sponsorship Contributions: Rs.", sponsorship)
print("Total Income: Rs.", total_income)
```

```
print("Total Expenses: Rs.", expenses)
print("Final Financial Balance: Rs.", balance)
if balance > 0:
    print("Status: Profit")
elif balance < 0:
    print("Status: Loss")
else:
    print("Status: Break-even")
```

5.Sample Input

```
Enter number of registrations: 4
Enter registration ID: 101
Enter registration ID: 102
Enter registration ID: 101
Enter registration ID: 103
Enter number of sponsorships: 2
Enter sponsorship amount: 2000
Enter sponsorship amount: 1000
Enter number of expenses: 2
Enter expense amount: 1200
Enter expense amount: 800
```

6. Sample Output

```
--- FESTIVAL AUDIT REPORT ---
Valid Participants: 3
Duplicate Registrations: ['101']
Registration Income: Rs. 1500
Sponsorship Contributions: Rs. 3000.0
Total Income: Rs. 4500.0
Total Expenses: Rs. 2000.0
Final Financial Balance: Rs. 2500.0
Status: Profit
```

Test Case 8

81. Campus Event Seating and Entry Validation Using Python

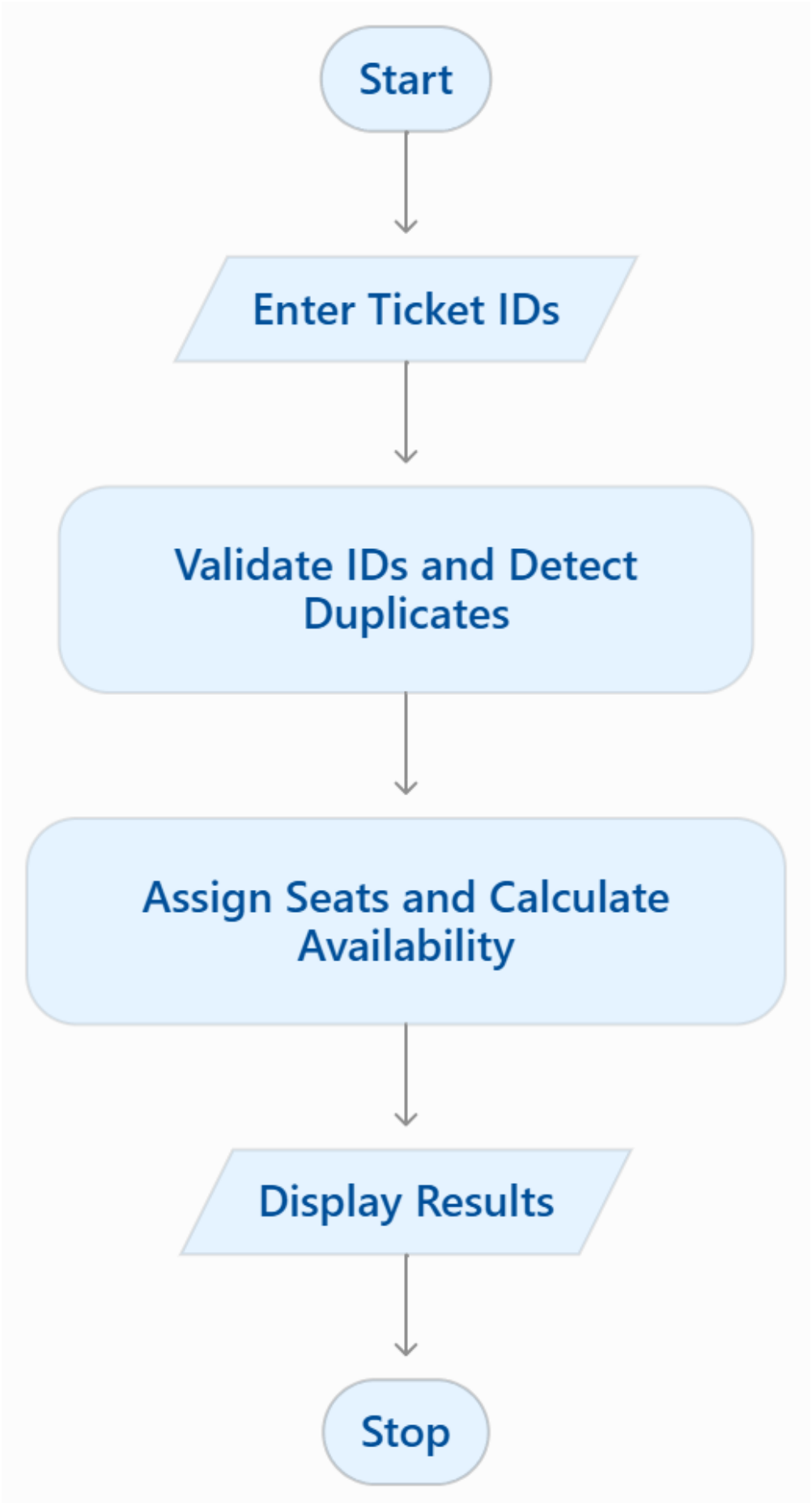
1. Problem Statement

Write a Python program to assign seats to registered participants using nested lists. Validate tickets using registration IDs, detect duplicate entries, and calculate the remaining seating capacity.

2. Algorithm

1. Start.
2. Create a nested list of seats and registration IDs.
3. Enter the ticket registration ID.
4. Validate the ID against registered participants.
5. Reject invalid IDs and detect duplicate entries.
6. Assign a seat to each valid entry.
7. Calculate available seating capacity.
8. Display seat assignments, duplicate entries, and remaining seats.
9. Stop.

3. Flowchart


Start
Enter Ticket IDs
Validate IDs and Detect Duplicates
Assign Seats and Calculate Availability
Display Results
Stop

4.Source Code

```
registrations = ["101", "102", "103", "104", "105", "106"]
seats = [[None, None, None],
        [None, None, None]]
entered = []
duplicates = []
invalid = []
n = int(input("Enter number of tickets to check: "))
for i in range(n):
    ticket = input("Enter registration ID: ")
    if ticket not in registrations:
        invalid.append(ticket)
    elif ticket in entered:
        duplicates.append(ticket)
    else:
        entered.append(ticket)
index = 0
for row in range(2):
    for col in range(3):
        if index < len(entered):
            seats[row][col] = entered[index]
            index += 1
available = 6 - len(entered)
print("\n--- EVENT SEATING REPORT ---")
print("Seat Assignments:")
for i, row in enumerate(seats, start=1):
    print("Row", i, ":", row)
print("Valid Entries:", len(entered))
```

print("Duplicate Entries:", duplicates)

print("Invalid Tickets:", invalid)

print("Available Seats:", available)

5. Sample Input

```
Enter number of tickets to check: 5
Enter registration ID: 101
Enter registration ID: 102
Enter registration ID: 101
Enter registration ID: 110
Enter registration ID: 103
```

6.Sample Output

```
--- EVENT SEATING REPORT ---
Seat Assignments:
Row 1 : ['101', '102', '103']
Row 2 : [None, None, None]
Valid Entries: 3
Duplicate Entries: ['101']
Invalid Tickets: ['110']
Available Seats: 3
```

Test Case 9

90. Integrated Campus Energy and Water Audit Using Python

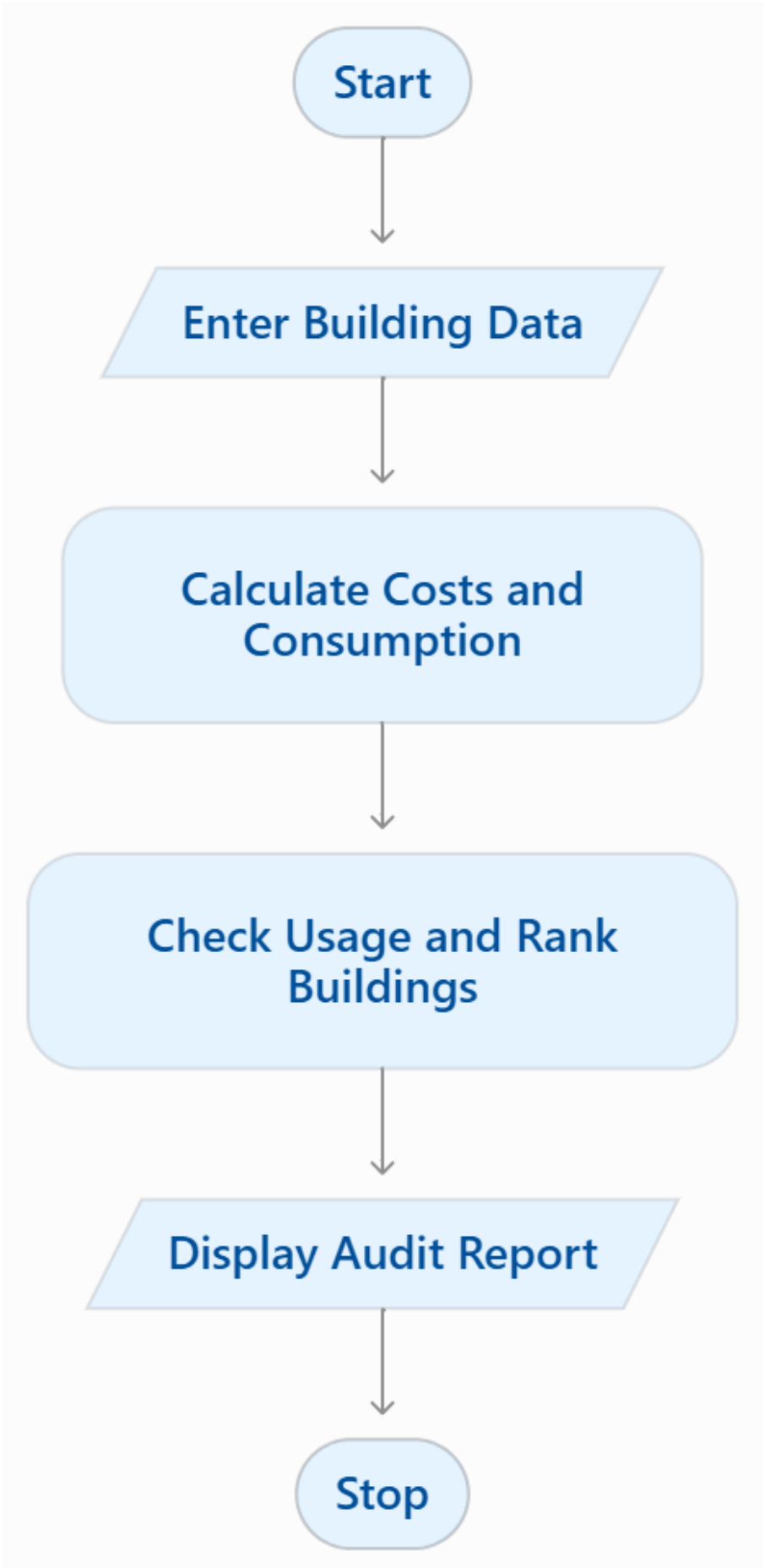
1. Problem Statement

90.Write a Python program to analyze electricity and water consumption across multiple campus buildings over two days. Calculate consumption costs and percentage changes, identify buildings with excessive consumption, and rank buildings using a scoring method.

2. Algorithm

1. Start.
2. Enter building names and electricity and water consumption for two days.
3. Calculate electricity and water costs.
4. Calculate percentage changes from Day 1 to Day 2.
5. Identify buildings exceeding consumption limits.
6. Calculate scores and rank buildings.
7. Display costs, percentage changes, excessive consumption alerts, and rankings.
8. Stop.

3. Flowchart


Start
Enter Building Data
Calculate Costs and Consumption
Check Usage and Rank Buildings
Display Audit Report
Stop

4.Source Code

```
n = int(input("Enter number of buildings: "))
buildings = []
for i in range(n):
    name = input("\nEnter building name: ")
    e1 = float(input("Day 1 electricity units: "))
    e2 = float(input("Day 2 electricity units: "))
    w1 = float(input("Day 1 water litres: "))
    w2 = float(input("Day 2 water litres: "))
    if min(e1, e2, w1, w2) < 0:
        print("Invalid consumption values!")
        break
    cost1 = e1 * 8 + w1 * 2
    cost2 = e2 * 8 + w2 * 2
    change = ((cost2 - cost1) / cost1 * 100
              if cost1 != 0 else 0)
    score = e2 + w2 / 100
    excessive = e2 > 100 or w2 > 500
    buildings.append({
        "name": name,
        "cost": cost2,
        "change": change,
        "score": score,
        "excessive": excessive
    })
else:
    print("\n--- CAMPUS AUDIT REPORT ---")
    for b in buildings:
```

```
        print("\nBuilding:", b["name"])
        print("Day 2 Total Cost: Rs.", round(b["cost"], 2))
        print("Cost Change:", round(b["change"], 2), "%")
        print("Score:", round(b["score"], 2))
        print("Consumption:",
              "Excessive" if b["excessive"] else "Normal")
    buildings.sort(key=lambda b: b["score"], reverse=True)
    print("\n--- BUILDING RANKING ---")
    for rank, b in enumerate(buildings, start=1):
        print(rank, b["name"], "- Score:", round(b["score"], 2))
```

5. Sample Input

```
Enter number of buildings: 2

Enter building name: Admin
Day 1 electricity units: 80
Day 2 electricity units: 120
Day 1 water litres: 400
Day 2 water litres: 600

Enter building name: Library
Day 1 electricity units: 60
Day 2 electricity units: 70
Day 1 water litres: 300
Day 2 water litres: 350
```

6. Sample Output

```
--- CAMPUS AUDIT REPORT ---

Building: Admin
Day 2 Total Cost: Rs. 2160.0
Cost Change: 50.0 %
Score: 126.0
Consumption: Excessive

Building: Library
Day 2 Total Cost: Rs. 1260.0
Cost Change: 16.67 %
Score: 73.5
Consumption: Normal

--- BUILDING RANKING ---
1 Admin - Score: 126.0
2 Library - Score: 73.5
```

Test Case 10

99. Integrated E-Commerce Order Processing and Reconciliation Using Python

1. Problem Statement

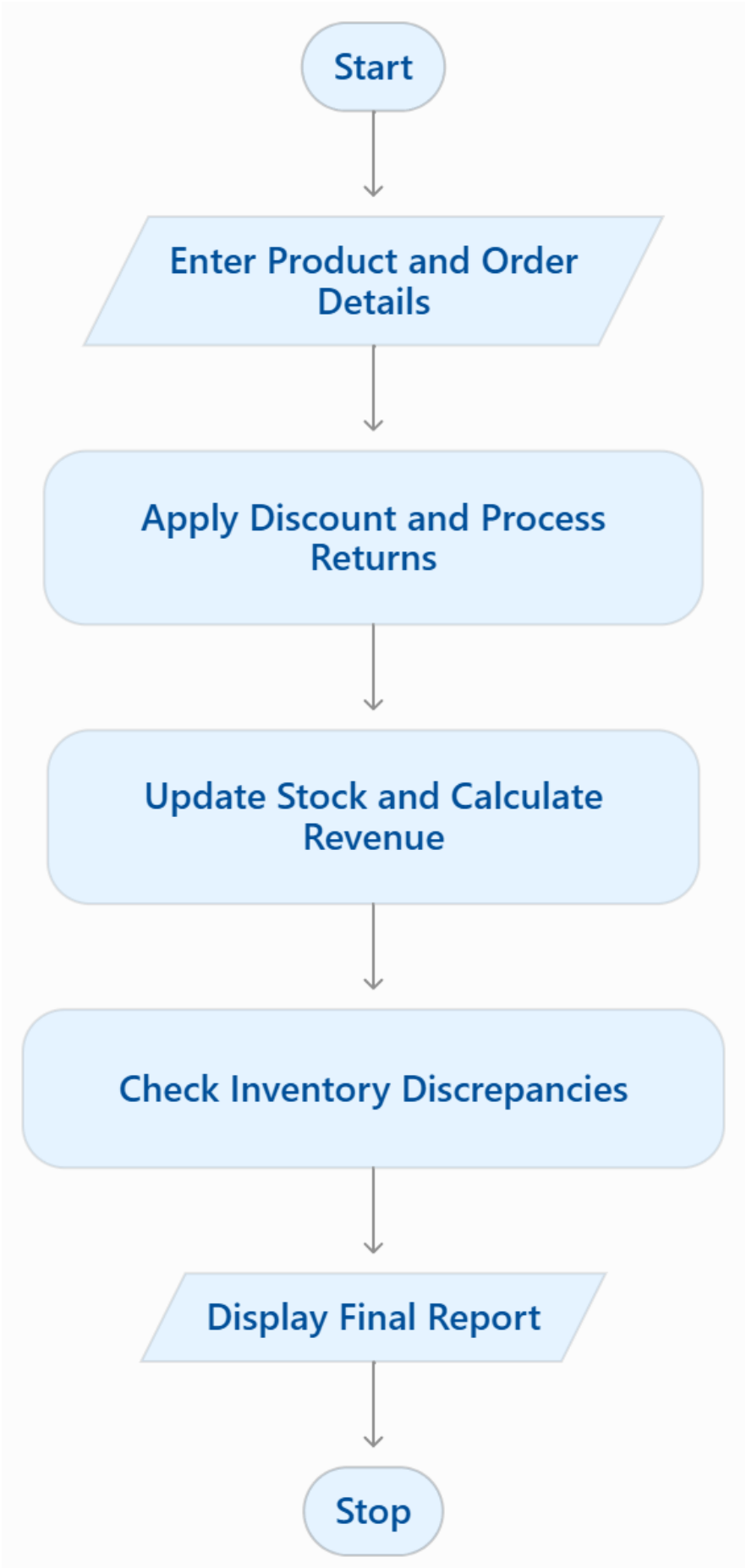
99.Write a Python program to process e-commerce orders, apply discounts, manage dispatches and returns, update stock quantities, calculate revenue, identify discrepancies, and generate a final inventory report.

2. Algorithm

1. Start.
2. Enter product details, prices, stock, and order quantities.
3. Calculate order values and apply discounts.

4. Process dispatches and returns.
5. Update stock and calculate net revenue.
6. Compare expected stock with actual stock.
7. Display revenue, discrepancies, and final inventory.
8. Stop.

3. Flowchart


Start
Enter Product and Order Details
Apply Discount and Process Returns
Update Stock and Calculate Revenue
Check Inventory Discrepancies
Display Final Report
Stop

4.Source Code

```
n = int(input("Enter number of products: "))
products = []
total_revenue = 0
for i in range(n):
    print("\nEnter details for product", i + 1)
    name = input("Product name: ")
    price = float(input("Price: "))
    stock = int(input("Initial stock: "))
    ordered = int(input("Ordered quantity: "))
    dispatched = int(input("Dispatched quantity: "))
    returned = int(input("Returned quantity: "))
    actual_stock = int(input("Actual stock: "))

    if (price < 0 or min(stock, ordered, dispatched,
                         returned, actual_stock) < 0
            or dispatched > ordered
            or returned > dispatched
            or dispatched > stock):
        print("Invalid product or order details!")
        break
    order_value = price * ordered
    discount_rate = 0.10 if order_value >= 5000 else 0
    discounted_price = price * (1 - discount_rate)
    revenue = (dispatched - returned) * discounted_price
    total_revenue += revenue
    expected_stock = stock - dispatched + returned
    discrepancy = actual_stock - expected_stock
```

```
    products.append({
        "name": name,
        "revenue": revenue,
        "expected": expected_stock,
        "actual": actual_stock,
        "discrepancy": discrepancy
    })
else:
    print("\n--- E-COMMERCE AUDIT REPORT ---")
    print("Total Net Revenue: Rs.", round(total_revenue, 2))
    for p in products:
        print("\nProduct:", p["name"])
        print("Net Revenue: Rs.", round(p["revenue"], 2))
        print("Expected Stock:", p["expected"])
        print("Actual Stock:", p["actual"])

        if p["discrepancy"] == 0:
            print("Inventory: Matched")
        else:
            print("Discrepancy:", p["discrepancy"], "units")
    print("\n--- FINAL INVENTORY SUMMARY ---")
    for p in products:
        print(p["name"], ":", p["expected"], "units expected")
```

5. Sample Input

```
Enter number of products: 2

Enter details for product 1
Product name: Laptop
Price: 40000
Initial stock: 10
Ordered quantity: 1
Dispatched quantity: 1
Returned quantity: 0
Actual stock: 9

Enter details for product 2
Product name: Mouse
Price: 600
Initial stock: 20
Ordered quantity: 10
Dispatched quantity: 8
Returned quantity: 2
Actual stock: 13
```

6. Sample Output

```
--- E-COMMERCE AUDIT REPORT ---
Total Net Revenue: Rs. 39240.0

Product: Laptop
Net Revenue: Rs. 36000.0
Expected Stock: 9
Actual Stock: 9
Inventory: Matched

Product: Mouse
Net Revenue: Rs. 3240.0
Expected Stock: 14
Actual Stock: 13
Discrepancy: -1 units

--- FINAL INVENTORY SUMMARY ---
Laptop : 9 units expected
Mouse : 14 units expected
```